

JVM и Android

Как выполняется код

Платформа, байткод и управление памятью

Разработка мобильных приложений на Android

Платформа перед изучением Kotlin



Мы будем писать на Kotlin

Почему Android Studio всё равно нужен JDK?

И нужно ли устанавливать Java на телефон?

Ответ требует разделить сборку проекта и работу приложения

Что называют Java

Термин	Что он обозначает
Язык Java	Правила записи исходного кода
Платформа Java	Среду исполнения, библиотеки и спецификации
JVM	Машину исполнения class-файлов
Команда java	Программу запуска JVM-приложения

Язык, инструменты и среда исполнения связаны, но имеют разные задачи

Краткая история Java

Период	Событие
Начало 1990-х	Джеймс Гослинг и команда Sun создают Oak
1995–1996	Публичный выход Java, затем JDK 1.0
2006–2007	Открытие исходников и развитие OpenJDK
2010	Oracle приобретает Sun Microsystems

Переносимость между устройствами была одной из исходных целей

JVM стала платформой для разных языков



Kotlin



Scala



Clojure



Groovy

Языки могут компилироваться в class-файлы
и использовать библиотеки общей платформы.

Kotlin умеет работать с JVM и имеет другие целевые платформы

Java, Android и Kotlin

Год	Что изменилось
2008	Первые Android-устройства. Приложения используют Dalvik
2014	Android 5.0: ART становится основной средой исполнения
2017	Google объявляет официальную поддержку Kotlin
2019	Android-разработка становится Kotlin-first

Выбор Kotlin сохраняет связь с Java API и инструментами JVM

Среда исполнения внутри Android

Упрощённая схема платформы

Приложения

Наше приложение и другие установленные программы

Android Framework

Готовые API: окна, уведомления, камера, местоположение

ART · библиотеки · HAL

Исполнение кода и доступ к оборудованию

Ядро Linux

Процессы, память, драйверы и базовые механизмы защиты

Оборудование

Процессор, экран, камера, датчики

JVM: виртуальная машина Java

- Загружает и проверяет class-файлы
- Выполняет инструкции и управляет объектами
- Работает внутри процесса операционной системы
- Реализации JVM: например, HotSpot и Eclipse OpenJ9

JVM предоставляет среду для программы, а эмулятор имитирует устройство

JVM, JRE и JDK

Термин	Расшифровка	Задача
JVM	Java Virtual Machine	Исполнение class-файлов
JRE	Java Runtime Environment	JVM и библиотеки для запуска
JDK	Java Development Kit	Среда и инструменты разработки

На компьютере разработчика нужен JDK

Что находится в JDK

Инструмент	Для чего нужен
<code>javac</code>	Компилирует Java-код в class-файлы
<code>java</code>	Запускает JVM-приложение
<code>javap</code>	Показывает содержимое class-файла
<code>jar</code>	Работает с архивами классов и ресурсов

Компилятор Kotlin поставляется отдельно от JDK

Кто поставляет JDK

Поставщик или проект	Дистрибутив
JetBrains	JetBrains Runtime, JBR
Eclipse Adoptium	Eclipse Temurin
Oracle	Oracle JDK и сборки OpenJDK
Amazon / Azul / Microsoft	Corretto / Zulu / Microsoft Build of OpenJDK

Android Studio включает JBR, основанный на OpenJDK

JVM на компьютере разработчика

Android Studio

Работает на JVM. JBR поставляется вместе со средой.

Gradle

Запускает сборку, подключает плагины и зависимости.

Компиляторы

Переводят исходники в следующий формат представления кода.

JDK для сборки и ART на телефоне живут в разных окружениях

Путь обычной Java-программы

Этап	Что получаем
1. Исходный код	Hello.java, текст программы
2. Компилятор javac	Hello.class, байткод и метаданные
3. Среда JVM	Загрузка и выполнение программы
4. Процессор	Выполнение машинных инструкций

Компиляция исходного кода и запуск программы происходят отдельно

Одна задача на Java и Kotlin

Java: Hello.java

```
public class Hello {  
    public static void main(  
        String[] args) {  
        System.out.println("Hello!");  
    }  
}
```

Kotlin: Hello.kt

```
fun main() {  
    println("Hello!")  
}
```

Разная запись программы может приводить к общему формату class-файлов

Компиляция и запуск на компьютере

```
javac Hello.java
```

```
java Hello
```

```
Hello!
```

javac создаёт
Hello.class.

java запускает
класс Hello.

Этот пример показывает обычный запуск JVM-программы

Что содержит class-файл

- Версию бинарного формата
- Описание класса, полей и методов
- Инструкции байткода для методов
- Константы и служебные метаданные

javap -с Hello показывает инструкции в читаемом виде

Исходный код и байткод JVM

Метод на Java

```
static int sum(int a, int b) {  
    return a + b;  
}
```

Фрагмент вывода javap -c

```
0: iload_0  
1: iload_1  
2: iadd  
3: ireturn
```

Байткод описывает операции машины JVM

Как JVM выполняет программу

Интерпретатор

Выполняет операции байткода через собственный машинный код.

JIT-компилятор

Во время работы создаёт машинный код для часто исполняемых участков.

Прогрев

Среда собирает сведения о выполнении и использует их для оптимизаций.

В итоге процессор всегда выполняет машинные инструкции

Почему Kotlin связан с Java

- Kotlin/JVM компилирует код в class-файлы
- Kotlin-код может вызывать Java-классы и библиотеки
- Java и Kotlin могут находиться в одном проекте
- Для Android остаются ограничения API и инструментов сборки

Совместимость с Java помогает использовать существующую экосистему

Путь кода в Android-приложении

Этап	Результат
Kotlin- или Java-компилятор	Исходники превращаются в class-файлы
D8, либо R8 при оптимизации	Код приложения и библиотек становится DEX
Упаковка и подпись	DEX, ресурсы и manifest попадают в APK
Запуск на устройстве	ART исполняет DEX-код приложения

В Android между class-файлами и запуском появляется этап DEX

Class-файлы и DEX

Свойство	class	DEX
Назначение	Формат JVM	Формат исполнения Android
Содержимое	Описание класса и его кода	Код и описания множества классов
Кто исполняет	JVM	ART, исторически Dalvik
Где встречается	Каталоги сборки, JAR	classes.dex внутри APK

Переименование .class в .dex не преобразует формат

D8 и R8

D8

Переводит байткод JVM в DEX.

Участвует в обычной Android-сборке.

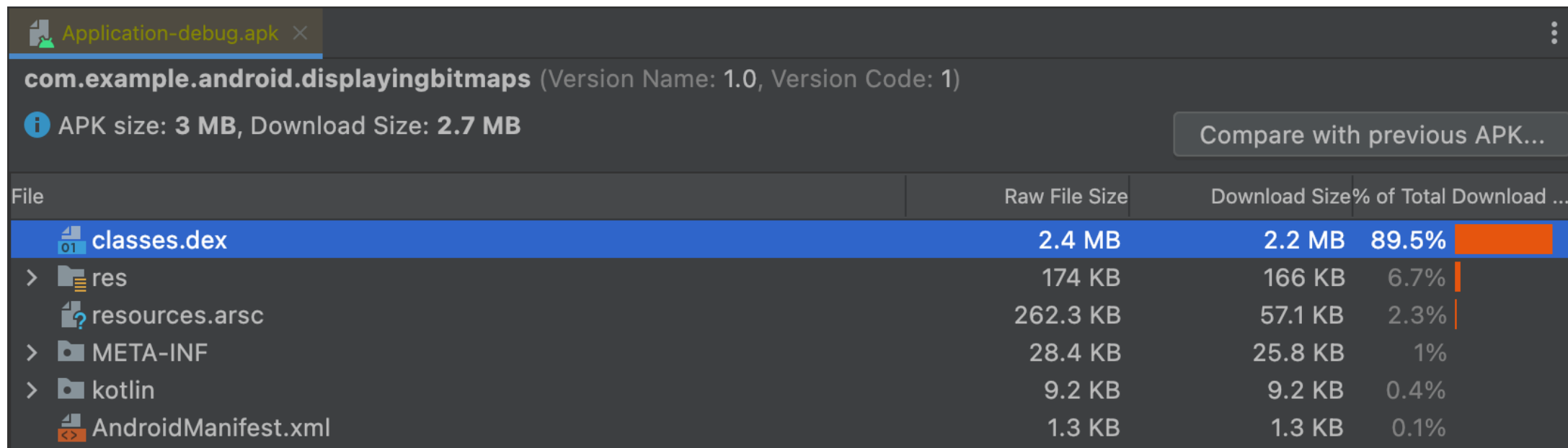
R8

Оптимизирует код, удаляет недостижимое и сокращает имена.

Также создаёт DEX.

Оба инструмента работают во время сборки на компьютере

Что находится внутри APK



The screenshot shows the 'APK Analyzer' window in Android Studio. At the top, it displays the package name 'com.example.android.displayingbitmaps' and version information. Below this, it states the total 'APK size: 3 MB' and 'Download Size: 2.7 MB'. A table lists the individual files and their sizes. The 'classes.dex' file is the largest, taking up 89.5% of the download size. Other files include 'res' (6.7%), 'resources.arsc' (2.3%), 'META-INF' (1%), 'kotlin' (0.4%), and 'AndroidManifest.xml' (0.1%).

File	Raw File Size	Download Size	% of Total Download ...
01 classes.dex	2.4 MB	2.2 MB	89.5%
> res	174 KB	166 KB	6.7%
resources.arsc	262.3 KB	57.1 KB	2.3%
> META-INF	28.4 KB	25.8 KB	1%
> kotlin	9.2 KB	9.2 KB	0.4%
AndroidManifest.xml	1.3 KB	1.3 KB	0.1%

classes.dex
Код приложения

res и assets
Ресурсы

AndroidManifest.xml
Описание приложения

APK Analyzer позволяет посмотреть результат сборки

JAR, APK и AAB

Формат	Для чего нужен
JAR	Архив JVM-классов и ресурсов. Часто библиотека
APK	Пакет, который Android устанавливает на устройство
AAB	Формат публикации. Из него получают APK для устройств

JAR с обычной JVM-программой нельзя просто установить как Android-приложение

ART: Android Runtime

- Исполняет код в формате DEX
- Управляет объектами и сборкой мусора
- Использует интерпретацию и компиляцию
- Работает вместе с Android framework и системными библиотеками

ART входит в Android. Пользователь не устанавливает JDK на телефон

Как Dalvik сменился на ART

Период	Среда исполнения
Ранний Android	Dalvik. Формат DEX уже существует
Android 4.4	ART доступна как экспериментальная альтернатива
Android 5.0	ART становится основной средой
Android 7.0 и далее	Сочетание интерпретации, JIT и AOT в ART

Название DEX сохранилось после смены среды исполнения

JVM и ART: сравнение

Вопрос	Обычная JVM	ART
Входной формат	class-файлы	DEX
В нашем курсе	Инструменты на компьютере	Код приложения на Android
API окружения	Java SE и библиотеки	Android API и доступные Java API
Управление памятью	Сборка мусора	Сборка мусора

Общие идеи исполнения не делают JVM и ART одной реализацией

Интерпретация, JIT и AOT в ART

Подход	Когда выполняется работа
Интерпретация	Runtime выполняет операции байткода
JIT, Just-In-Time	Компилирует участки во время работы приложения
AOT, Ahead-Of-Time	Компилирует участки заранее, до их выполнения
Профили	Помогают выбрать важные участки для оптимизации

ART сочетает подходы с учётом версии Android и профилей приложения

Нативный код в Android

Kotlin и Java

Обычная сборка приложения создаёт DEX-код.

ART обеспечивает управляемое исполнение.

C и C++

NDK создаёт нативные библиотеки, обычно .so.

Сборка учитывает ABI целевого процессора.

Одно приложение может объединять управляемый и нативный код

Версии Android в настройках проекта

Настройка	Смысл
compileSdk	Какие Android API доступны при компиляции
minSdk	Минимальный API level устройства для приложения
targetSdk	Под какое поведение Android приложение адаптировано

Версия JDK на ноутбуке задаётся отдельно

Совместимость кода с устройством

- Новый JDK на ноутбуке не обновляет Android на телефоне
- Версия JVM-байткода задаёт формат промежуточных class-файлов
- D8 и R8 должны поддерживать результат работы компиляторов
- Вызванный API должен существовать на устройстве или иметь замену

Поддержку проверяют для всей цепочки сборки и исполнения

Desugaring: адаптация к старым Android

- Инструменты переписывают некоторые конструкции в совместимую форму
- Core library desugaring добавляет поддержку части Java API
- Доступность зависит от API, настроек и версий инструментов
- Новые возможности самой ОС всё равно требуют проверки

Desugaring расширяет совместимость в определённых границах

Какие библиотеки видит приложение

Слой	Примеры
Kotlin standard library	Функции и типы, удобные для Kotlin
Java API в Android	Коллекции, строки и другие поддерживаемые API
Android framework	Activity, доступ к системным возможностям
Jetpack и внешние библиотеки	Compose, Room, сетевые и другие библиотеки

Перед подключением JVM-библиотеки проверяют поддержку Android

Garbage Collection: сборка мусора

- Программа создаёт объекты и массивы в управляемой памяти
- Ссылки позволяют программе обращаться к объектам
- GC определяет, какие объекты стали недостижимыми
- Память недостижимых объектов можно использовать повторно

Автоматическая сборка памяти не отменяет управление временем жизни данных

Как GC определяет, что сохранить

Ситуация	Что видит сборщик
Экран хранит ссылку на изображение	Объект достижим и должен оставаться
Две переменные ссылаются на один объект	Удаления одной ссылки недостаточно
Ни одной цепочки от корней GC не осталось	Объект можно собрать
Закрытый экран остался в глобальном списке	Он всё ещё достижим

«Больше не нужен пользователю» и «недостижим» могут не совпадать

Почему закрытый экран остался в памяти?

Приложение сохраняет каждый открытый экран в общий список. Список никогда не очищается.

Пользователь закрыл десять экранов. Почему GC не удалил их?

Какая ссылка удерживает данные и когда её нужно убрать?

Память и отзывчивость Android-приложения

- Большое количество новых объектов создаёт работу для GC
- Паузы и затраты CPU могут влиять на плавность интерфейса
- Удержание ненужных объектов может привести к OutOfMemoryError
- Файлы и сетевые соединения требуют своевременного закрытия

Сборка мусора внутри процесса и завершение процесса системой различаются

На каком этапе возникла ошибка?

Этап	Пример
Запуск сборки	Gradle или плагину не подходит выбранный JDK
Компиляция	Ошибка типа или ссылка на неизвестное имя
Сборка и установка	Конфликт зависимостей или неподходящий minSdk
Работа приложения	Нет нужного API, закончилась память, неверные данные

Сообщение об ошибке сначала связывают с этапом выполнения

Мини-задача: собрать картину целиком

**Ноутбук: Kotlin, JDK, Gradle.
Телефон: установленный APK.**

Что преобразует исходник?

Где появляются class и DEX?

Что исполняет приложение на телефоне?

Можно ли добавить новое Android API, просто обновив JDK?

Разбор мини-задачи

Часть цепочки	Ответ
Исходный Kotlin-код	Kotlin-компилятор создаёт class-файлы
Android-сборка	D8 или R8 создаёт DEX, сборка упаковывает APK
Устройство	Android запускает процесс, ART исполняет DEX
Новый API Android	Зависит от версии ОС и библиотек, а не только JDK

Java нужна инструментам, связь с JVM остаётся и при выборе Kotlin

Демонстрация после лекции

- Посмотреть версию JDK, которую использует проект
- Скомпилировать Hello.java и выполнить `javap -c Hello`
- Открыть готовый APK в APK Analyzer и найти `classes.dex`
- Запустить готовое приложение на устройстве или эмуляторе

Связываем исходный код, результат сборки и работающий процесс

Что нужно унести с занятия

- JDK и JVM участвуют в работе инструментов на компьютере
- Kotlin и Java могут приводить к одному формату class-файлов
- Android-сборка создаёт DEX, который исполняет ART
- Версии JDK, байткода и Android отвечают за разные вещи
- GC сохраняет объекты, которые приложение продолжает удерживать

Следующая пара: язык Kotlin и первые самостоятельные программы

Источники и дальнейшее чтение

- Android Developers: Java versions in Android builds
- AOSP: ART, DEX и способы исполнения кода
- Android Developers: память, SDK и APK Analyzer
- Спецификация JVM и документация Kotlin

Робот Android: Google, CC BY 3.0. Логотипы языков: Devicon.

Ссылки на материалы находятся в заметках соответствующих слайдов